



Computer Organization and Assembly language

Course: Computer Organization and Assembly Language (Lab) - CS-318

1. AIM

To study and implement **PUSH**, **POP**, and **nested subroutines** in 8086 assembly language using EMU8086.

In 8086 assembly, the **stack** is a reserved area of memory used to temporarily store **data, return addresses, and register values** in a **Last-In, First-Out (LIFO)** manner.

- **PUSH** → places data onto the stack
- **POP** → removes data from the stack
- **CALL/RET** → use stack to store return addresses

The **SP (Stack Pointer)** register points to the top of the stack.

2. THEORY

2.1 PUSH Instruction

- PUSH stores (pushes) data onto the **stack**.
- Stack grows **downward** (toward lower memory).
- SP decreases by 2 after each PUSH.

Format:

PUSH register

PUSH memory

2.2 POP Instruction

- POP retrieves (pops) data from the **stack**.
- Stack shrinks upward.
- SP increases by 2 after each POP.

Format:

POP register

POP memory

2.3 NESTED SUBROUTINES

A nested subroutine occurs when:

- Main program calls Subroutine A
- Subroutine A calls Subroutine B
- Subroutine B finishes → returns to A
- Subroutine A returns → back to main

Uses multiple CALL and RET instructions.

Proper stack usage is important.

3. PURPOSE OF USING PUSH/POP & NESTING

- Helps save register values
- Allows subroutines to use temporary data
- Ensures correct return addresses
- Supports modular and hierarchical programming

4. ALGORITHM

1. Start program.
2. Initialize required registers.
3. Push data onto stack.

4. Call main subroutine.
5. Inside subroutine, call another subroutine (nested).
6. Pop data from stack.
7. Return to the main program.
8. Exit

5. PROGRAM 1 – PUSH & POP

Program: Save two registers, modify them, then restore.

```
org 100h
```

```
mov ax, 1234h
```

```
mov bx, 5678h
```

```
push ax
```

```
push bx
```

```
mov ax, 0000h
```

```
mov bx, 0000h
```

```
pop bx
```

```
pop ax
```

```
mov ah, 4Ch
```

```
int 21h
```

Purpose:

AX and BX return to their original values.

6. PROGRAM 2 – Nested Subroutine

Description:

Main → calls SUB1 → SUB1 calls SUB2.

```
org 100h
```

```
call sub1
```

```
mov ah, 4Ch
```

```
int 21h
```

```
; ----- SUB1 -----
```

```
sub1 proc
```

```
    mov dl, 'A'
```

```
    mov ah, 02h
```

```
    int 21h
```

```
    call sub2    ; nested call
```

```
    mov dl, 'C'
```

```
    mov ah, 02h
```

```
    int 21h
```

```
ret
```

```
sub1 endp
```

```
; ----- SUB2 -----
```

```
sub2 proc
```

```
    mov dl, 'B'
```

```
    mov ah, 02h
```

```
    int 21h
```

```
ret
```

```
sub2 endp
```

Output:

ABC

7. PROGRAM 3 – Subroutine with PUSH/POP + Nested Calls

```
org 100h

mov cx, 5
call print_count

mov ah, 4Ch
int 21h

print_count proc
    push cx

loop1:
    call print_star
    loop loop1

    pop cx
ret
print_count endp
```

```
print_star proc
    mov dl, '*'
    mov ah, 02h
    int 21h
ret
print_star endp
```

Output:

LAB EXERCISE

- Write a program that uses PUSH and POP to save and restore register values before and after a subroutine call.
 - Write a program that uses PUSH to pass a value to a subroutine and POP to retrieve the modified result.
 - Write a program where the main program pushes two numbers, and a nested subroutine pops them and displays the sum.
-